Bibliothèques JavaScript

Lodash, jQuery & Axios : Les outils essentiels pour booster votre développement web.

SUPINFO 1WEBD — DAY 3

AU PROGRAMME

01 Introduction aux Bibliothèques

Pourquoi et quand utiliser des outils externes ?

02 Lodash : La boîte à outils utilitaire

Simplifier la manipulation de données complexes.

03 jQuery : Simplifier le DOM

Héritage, syntaxe et pertinence actuelle.

04 Axios : Maîtriser les requêtes HTTP

Communication client-serveur robuste et moderne.

05 Choisir le bon outil

Critères de sélection pour vos futurs projets.

POURQUOI UTILISER DES BIBLIOTHÈQUES ?

Gain de temps

Ne réinventez pas la roue. Utilisez des solutions éprouvées pour vous concentrer sur la logique métier de votre application.

Fiabilité

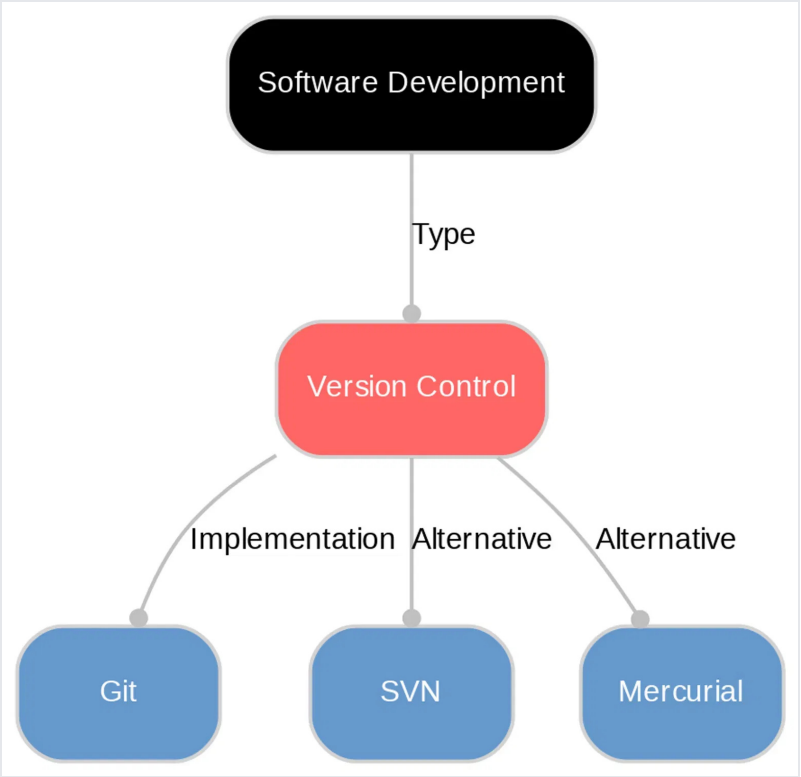
Bénéficiez de code testé par des millions de développeurs, réduisant ainsi les bugs et les failles de sécurité.

Lisibilité

Écrivez du code plus concis et expressif. Les bibliothèques offrent souvent des abstractions puissantes et intuitives.

Compatibilité

Gérez automatiquement les différences entre navigateurs sans avoir à écrire de code spécifique pour chacun.



LODASH : LA BOÎTE À OUTILS

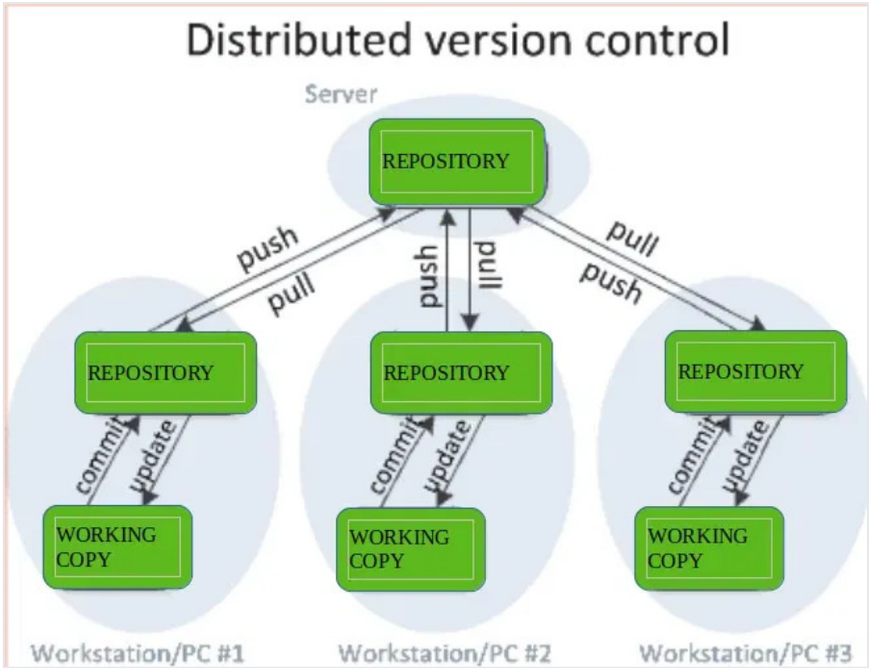
INTRODUCTION

Lodash est une bibliothèque JavaScript d'utilitaires qui simplifie la manipulation des tableaux, des objets et des chaînes de caractères. Elle permet d'écrire un code plus concis, lisible et performant.

POURQUOI L'UTILISER ?

Efficacité : Fonctions prêtes à l'emploi pour des tâches complexes.
Fiabilité : Évite les erreurs courantes de manipulation de données.
Performance : Algorithmes optimisés pour le traitement de gros volumes.

INSTALLATION



Lodash aide à gérer la complexité des flux de données.

LODASH EN ACTION

_.uniq

Supprime les doublons d'un tableau.

```
const arr = [1, 2, 1, 4, 1, 3];
const result = _.uniq(arr);
// result: [1, 2, 4, 3]
```

_.groupBy

Regroupe par critère spécifique.

```
const users = [{ 'age': 20 }, { 'age': 25 }, { 'age': 20 }];
const result = _.groupBy(users, 'age');
// result: { '20': [...], '25': [...] }
```

_.debounce

Limite la fréquence d'exécution.

```
const save = _.debounce(() => {
  console.log('Saved!');
}, 1000);
// Appelé après 1s d'inactivité
```

_.isEqual

Comparaison profonde d'objets.

```
const obj1 = { a: 1, b: { c: 2 } };
const obj2 = { a: 1, b: { c: 2 } };
const result = _.isEqual(obj1, obj2);
// result: true
```

JQUERY : LE GÉANT DU WEB

> **Philosophie**
>
> ***"Write less, do more"***

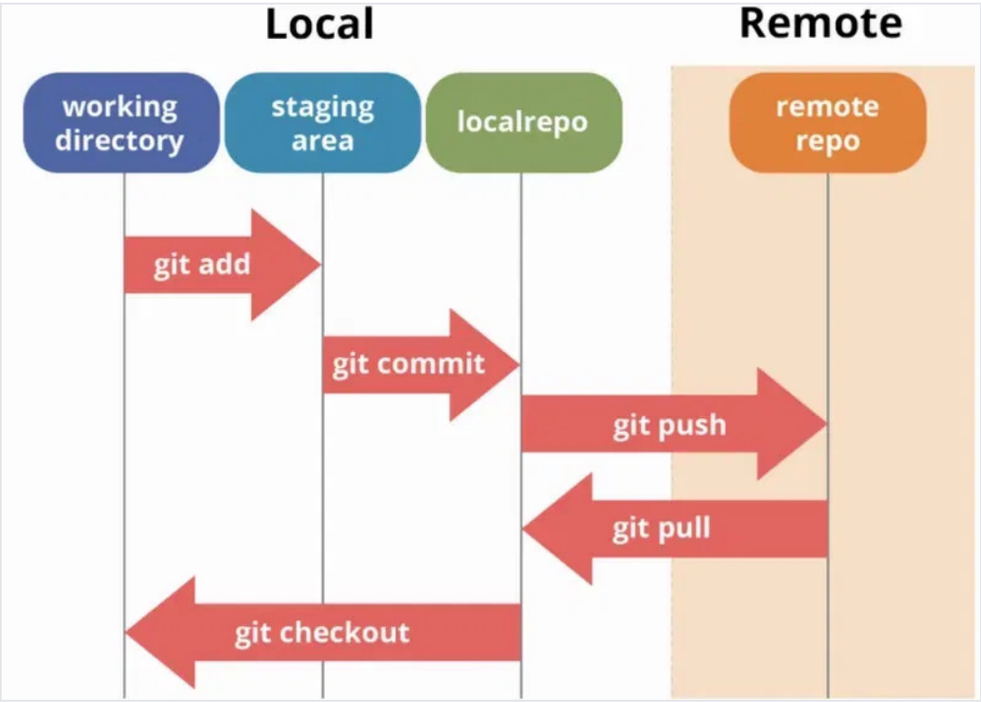
HISTOIRE

Créé en 2006 par John Resig, jQuery a révolutionné le développement web en simplifiant les interactions complexes.

CONCEPT CLÉ

Utilisation de la syntaxe CSS `$(selector)` pour manipuler le DOM, gérer les événements et créer des animations.

HÉRITAGE



JQUERY : SYNTAXE & DOM

JAVASCRIPT NATIF

```
document.querySelector('#id')
.innerHTML = 'Hello';
```

JQUERY

```
$('#id').html('Hello');
```

Manipulation

.append(), **.prepend()** : Ajout de contenu HTML de manière concise.

Événements

.on('click', fn) : Gestion d'événements simplifiée et robuste.

Animations

.fadeIn(), **.slideUp()** : Effets visuels prêts à l'emploi en une ligne.

JQUERY EN 2026 : TOUJOURS UTILE ?

Pourquoi l'utiliser ?

Prototypage Rapide

Idéal pour des petits projets ou des démonstrations où la simplicité prime sur la structure complexe.

Écosystème de Plugins

Accès à des milliers de plugins matures comme Slick (carrousels) ou DataTables (tableaux interactifs).

Maintenance

Essentiel pour maintenir des millions de sites web existants qui reposent encore sur cette technologie.

Pourquoi l'éviter ?

JavaScript Moderne (ES6+)

Le JS natif a intégré la plupart des fonctionnalités de jQuery (querySelector, fetch, classList).

Frameworks Réactifs

React, Vue et Angular offrent une gestion d'état et du DOM bien plus performante pour les apps complexes.

Poids de la Lib

Ajouter 30kb+ pour des fonctionnalités déjà présentes nativement impacte inutilement le temps de chargement.

> *"jQuery n'est pas mort, il est devenu une partie intégrante de l'histoire du web, mais son usage pour de nouveaux projets doit être mûrement réfléchi."*

AXIOS : COMMUNICATION MODERNE

> Axios est une bibliothèque JavaScript utilisée pour effectuer des requêtes HTTP, basée sur les **Promesses (ES6)**.

POLYVALENCE

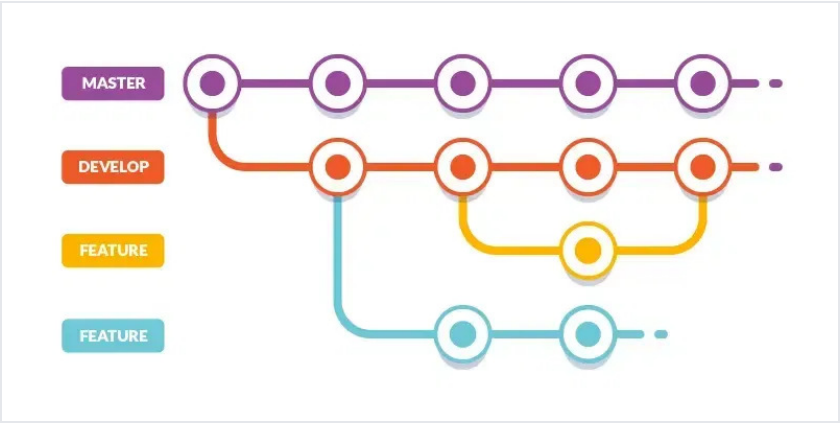
Fonctionne de manière identique côté navigateur et côté serveur avec Node.js.

ROBUSTESSE

Simplifie la gestion des erreurs, des timeouts et permet l'utilisation d'intercepteurs.

AUTOMATISATION

Gère automatiquement la transformation des données en format JSON.



Axios structure et sécurise vos échanges de données complexes.

AXIOS EN PRATIQUE

EXEMPLE DE REQUÊTE GET

```
axios.get('/api/user')
  .then(res => {
    console.log(res.data);
  })
  .catch(err => {
    console.error(err);
  });
```

FONCTIONNALITÉS AVANCÉES

Configuration Globale

Définissez une Base URL et des Headers par défaut pour toutes vos requêtes.

Intercepteurs

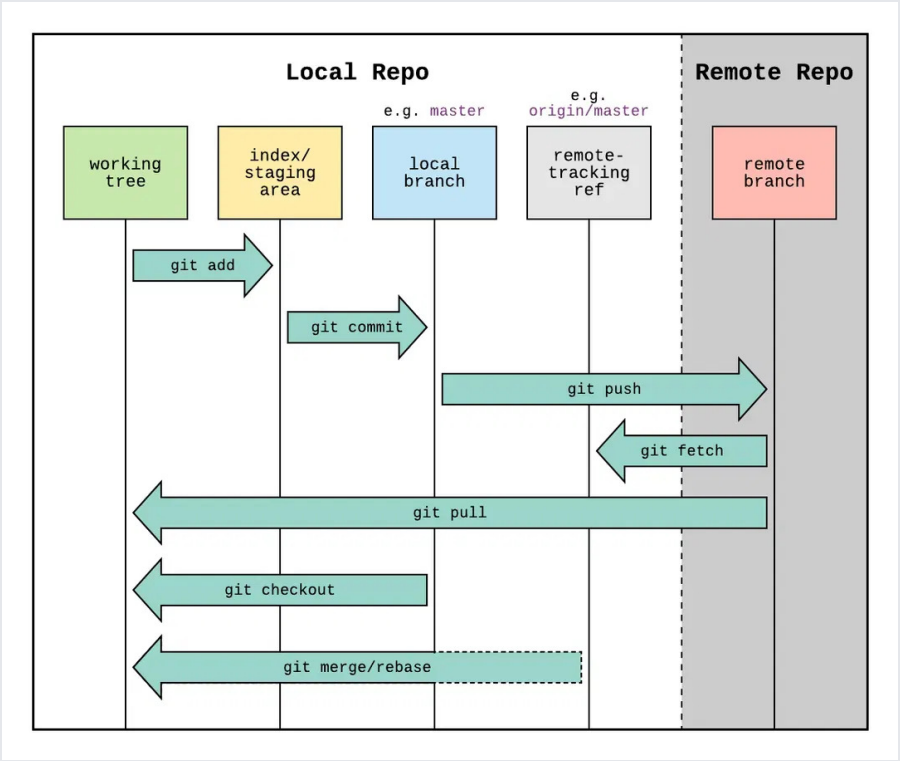
Transformez les requêtes ou réponses avant qu'elles ne soient traitées (ex: Auth).

Gestion des Timeouts

Annulez automatiquement les requêtes qui prennent trop de temps.

JSON Automatique

Axios transforme automatiquement les données en JSON pour vous.



Axios simplifie le flux de communication client-serveur.

AXIOS OU FETCH API ? LE MATCH

CARACTÉRISTIQUE	FETCH (NATIF)	AXIOS (LIB)
Traitement JSON	Manuel (via `.json()`)	**Automatique**
Gestion du Timeout	Complexe (AbortController)	**Natif et Simple**
Intercepteurs	Non (doit être simulé)	**Oui (Puissant)**
Support Navigateur	Navigateurs récents	**Universel (IE11+)**

BIEN CHOISIR SES OUTILS

01 Besoin Réel

Posez-vous la question : est-ce que le JavaScript natif (ES6+) peut accomplir la tâche sans bibliothèque externe ? Évitez la sur-ingénierie.

02 Poids & Performance

Chaque bibliothèque alourdit votre application. Vérifiez l'impact sur le temps de chargement (Bundle Size) et privilégiez les libs modulaires.

03 Maintenance & Santé

La bibliothèque est-elle activement maintenue ? Vérifiez la fréquence des mises à jour, le nombre de contributeurs et la communauté sur GitHub.

04 Écosystème

Existe-t-il une documentation claire ? Y a-t-il des plugins ou des extensions qui facilitent son intégration dans votre projet ?

> **Conseil d'expert :** Utilisez des outils comme *BundlePhobia* pour analyser le coût d'une bibliothèque avant de l'ajouter à votre projet.

CONCLUSION : POINTS CLÉS

1 **Multiplicateurs de Productivité**

Les bibliothèques ne sont pas des béquilles, mais des outils pour accélérer votre développement et fiabiliser votre code.

2 **Spécialisation des Outils**

Utilisez **Lodash** pour la manipulation complexe de données et **Axios** pour une communication réseau robuste.

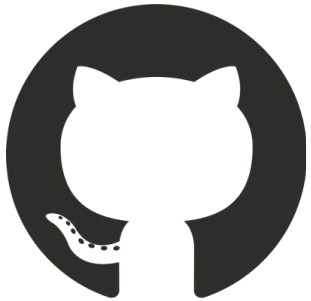
3 **Héritage et Modernité**

Gardez **jQuery** pour la maintenance de projets existants ou le prototypage ultra-rapide, mais privilégiez le natif pour le neuf.

"La meilleure bibliothèque est celle dont vous avez **réellement besoin**."

Questions ?

Merci de votre attention !



RESSOURCES

Lodash Documentation
`lodash.com/docs`

jQuery Learning Center
`learn.jquery.com`

Axios GitHub
`github.com/axios/axios`

MDN Web Docs
`developer.mozilla.org`